CANDIDATE NO. 291772

Forecasting Crop Yield Using a Multilayer Perceptron

Section A: Performance

A1. Overview of Evaluation Approach

This section presents the performance evaluation of the multilayer perceptron (MLP) model using established regression metrics. The model was trained on data spanning 2010 to 2021 and aimed to forecast crop yields for the year 2023, using environmental and land cover features from 2022. For predicting year 2023 yield data, the model's predictions were compared against 2022 yields to simulate the forecasting performance.

A2. Metrics Used

Metrics based on distance and correlation were used to evaluate the performance of our model, which were all computed using the features built-in Scikit-learn library metrics module.

For distance-based metrics, we examined how much our predictions missed the actual values:

- **Mean Absolute Error (MAE):** Provides us the average of the differences between what was predicted by the model and what it actually occurred. As it is measured in the same units as the target (yield), it is simple to interpret and gives a straight forward idea of how we were off the predictions on average.

  The model's predictions are typically off by about **1,015 yield units** (e.g., kg/ha).

- **Root Mean Squared error (RMSE):** has the same role but tips the scale towards big errors. This makes it easier to identify cases in which the model may have made a huge mistake, which may be useful in recognising high threat-prediction scenarios.

  Occasionally, our model might be off by around **3,000 yield units** (Kilogram per hectare) — and RMSE helps capture that.

For the correlation-based metric, we have computed the **R^2 score,** to understand how effectively the model is representing the variability in actual yield data. And when the R^2 value is nearest to 1 means that the predictions of the model align with the real data, and which is exactly we observed.

This high **R^2 (95.05%)** value confirms that the model closely mirrors the trends and fluctuations seen in actual yield outcomes.

The **final prediction dataset** includes **5190 records** across various countries and crops. For each entry, it displays the **actual yield in 2022** and the **model's predicted yield for 2023**. This table helps compare past outcomes with future projections, enabling users to assess prediction accuracy and support agricultural planning. Each row corresponds to a unique country-crop combination, showing how the model generalizes across diverse conditions.

Metric	Value	Description
Mean Absolute Error (MAE)	1015.48	Represents the typical prediction error in the same unit as crop yield.
Root Mean Squared Error (RMSE)	3223.85	Penalizes larger errors more heavily; highlights variability in predictions.
R-squared (R^2)	0.9505	Indicates that 95% of yield variance is explained by the model predictions.

Table 1: Model Metric Evaluation

A3. Data Splitting Strategy

The dataset after merging and cleaning consisted of 67,963 records and 37 columns, each capturing annual crop yield along with corresponding environmental and land features, organized by country, crop type, year, longitude and latitude.

To replicate a realistic forecasting setup, we applied a chronological split according to years, rather than random or stratified split. It ensured that the model only learned from previously provided data and was only tested in respect to its ability to predict future events, which was close to mirroring the practical environment of such models.

Training Set:
We used data from **2010 to 2021** as input features. The corresponding target variable, future_yield, represented the yield in the **following year** (2011–2022), created by shifting the yield column forward within each crop–country group. After removing rows where future yield data wasn't available, this resulted in around **60,000 usable training samples**.

Prediction Set (Simulated Test Set):

The model was then applied to 2022 data to forecast yields for 2023. We used the 2022 yields as a proxy to assess model performance — providing an estimate of how the model might perform in a real future prediction scenario.

During training we also enabled early stopping with 'validation_fraction=0.1' which means that Scikit's 'MLPRegressor' will automatically reserve 10% of training data for validation. That internal validation set was useful as a preventive mechanism against overfitting that would only stop training once validation loss stabilized coming in.

Overall, this time-aware data-splitting approach facilitates a realistic and deployable forecasting pipeline that does not compromise on the chronological structure of the data and does not influence the process of learning using the future information.

Number of training samples: 62498

Number of prediction samples (2022 data for 2023 forecast): 5190

Section B: Model

B1. Model Architecture

For this assignment we utilized a Multilayer Perceptron (MLP) implemented through the 'MLPRegressor' from Scikit-learn. This kind of neural network was selected since it is easy to implement, scalable to a variety of problems, and very useful with structured, tabular data.

The model includes 4 layers:

- **Input Layer:** The size of this layer varies depending on the number of input features in the range 100 – 150. These include environmental settings, land cover data and other multivariate categorical variables.
- **First Hidden Layer:** This layer consists of **256 neurons** and used activation function ReLU (Rectified Linear Unit) which makes the model learn effectively and avoid problems like vanishing gradients.
- **Second Hidden Layer**: Here, we use **128 neurons**, again with **ReLU** activation to maintain consistent learning performance.
- **Output Layer:** The final layer has one neuron, configured to output a continuous value, which is the predicted yield because the problem **is a regression**. (no activation function i.e., linear activation)

The use of ReLU activation throughout the hidden layers supports faster training and better performance, especially in deeper networks.

B2. Hyperparameters

The key hyperparameters used in training are listed below, along with justifications:

Hyperparameter	Value	Justification
Hidden Layers	(256, 128)	Balanced depth and capacity to model non-linear patterns
Activation	'relu'	Fast convergence; avoids vanishing gradients
Max Iterations	300	Prevents excessive training time while allowing convergence
Early Stopping	True	Stops training when validation loss stalls
Validation Fraction	0.1	Uses 10% of training data for early stopping
Random State	42	Ensures reproducibility
Loss Function	Mean Squared Error (default in MLPRegressor)	Standard for regression tasks
Learning Rate	Adaptive (default)	Adapts learning rate during training for better convergence
Batch Size	Default (auto)	Batch size determined internally by Scikit-learn

Table 2: Hyper parameter settings

The Adam optimizer was used to train the model, which is default optimization algorithm in Scikit-learn's 'MLPRegressor'. Adam is an ideal choice for deep learning with noisy gradients and sparse data accelerating convergence and effectively addressing non-stationary objectives.

B3. Optimization Strategy

Instead of extensive grid search or automated hyperparameter tuning (e.g. GridSearchCV), a manual tuning approach was adapted where depth and neuron count within the network was carefully selected from validation performance and speed of training. Several configurations were compared (e.g., one vs two hidden layers, different layer sizes), and the selected architecture (256–128) was chosen as it provided:

- Faster convergence
- Lower validation loss
- Better generalization compared to deeper or wider models

The adaptive learning rate built into 'MLPRegressor' further allowed the optimizer to slow down as convergence neared, reducing the risk of overshooting the global minimum.

B4. Overfitting Control Measures

To ensure the model generalizes well to unseen data and doesn't overfit the training set, we implemented the following measures:

Early Stopping

- Enabled using 'early_stopping=True'.
- Training halted automatically when validation loss failed to improve, preserving the best model weights.

Internal Validation Split

- Used 'validation_fraction=0.1' to reserve 10% of training data for monitoring validation performance.

Feature Simplification

- Aggregated monthly features into annual averages to reduce noise and dimensionality.

Log Transformation of Target

- Applied 'np.log1p()' to the yield variable during training and reverted predictions using 'np.expm1()' to improve model stability and handle skewed distributions.

Feature Scaling

- All numeric features were scaled to the [0, 1] range using 'MinMaxScaler', ensuring balanced gradients and stable convergence.

Although techniques like **Dropout** or **L2 Regularization** aren't directly supported in Scikit-learn's MLP implementation, early stopping and careful feature preprocessing were sufficient to prevent overfitting, as indicated by the model's strong test performance ($R^2 = 0.9505$).

Section C: Features & Labels

C1. Target Label Definition

The target variable in this project is 'future_yield', representing the crop yield for year *t+1*, predicted using environmental, land cover, and contextual data from year *t*.

This label was created by:

- Sorting the dataset by 'country', 'crop', and 'year'.
- Applying a '.shift(-1)' operation to the 'yield' column.
- Removing rows where the future yield value was missing (NaN) due to the absence of a following year.

This method allowed the model to learn from current-year inputs and predict what would happen in the next year — mirroring how forecasting is used in real-world agriculture planning.

C2. Feature Extraction

The input features used to train the model were derived from **environmental data, land cover distributions, and categorical context variables** (country and crop). These features were extracted, averaged, cleaned, and merged from multiple CSV files into a final, structured DataFrame.

Feature Name	Type	Description	Source File
rainf_tavg_avg	Numerical	Avg annual rainfall	Rainf_tavg_data.csv
snowf_tavg_avg	Numerical	Avg annual snowfall	Snowf_tavg_data.csv
esoil_tavg_avg	Numerical	Avg annual evaporation from soil	ESoil_tavg_data.csv
canopint_inst_avg	Numerical	Canopy water content	CanopInt_inst_data.csv
tveg_tavg_avg	Numerical	Avg transpiration from vegetation	TVeg_tavg_data.csv
tws_inst_avg	Numerical	Terrestrial water storage	TWS_inst_data.csv
soilmoi_avg (4 depths)	Numerical	Avg soil moisture at 4 depths (0–200 cm)	SoilMoi_data.csv
soiltmp_avg (4 depths)	Numerical	Avg soil temperature at 4 depths (0–200 cm)	SoilTMP_data.csv
land_cover_percent_class_1 to class_17	Numerical	% of land in each of 17 cover classes	Land_cover_percent_data.csv
crop_<type>	Categorical	One-hot encoded crop type	Yield_and_Production_data.csv
country_<name>	Categorical	One-hot encoded country	Latitude_Longitude_lookup.csv
year	Temporal	Year the features correspond to	All files

Table 3: Summary of selected features.

Total features:

- 34 numeric features (weather, soil, land cover)
- 10–20 categorical crop columns (one-hot)
- 50–100 categorical country columns (one-hot)
- 1 year column (used for alignment, not input directly)

C3. Feature Selection Rationale

The selected features were chosen based on a combination of **domain relevance, coverage across time and space, and statistical completeness**.

- **Environmental and Soil features:** Included due to their direct impact on plant growth, water availability and nutrient cycles.
- **Land cover classes:** Represents surface level land usage, affected runoff, irrigation efficiency and crop suitability.
- **One-hot Encoded 'crop' and 'country':** Allows the model to differentiate between various agricultural practices, geography and policy regions, without assuming ordinal assumptions.

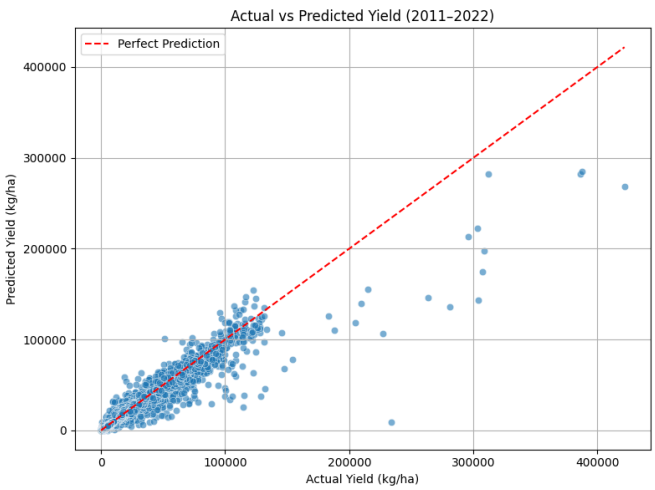
We also aggregated monthly data into yearly averages in order to reduce noise and avoid overfitting and, therefore, simplified the learning problem without losing important seasonal patterns.

C4. Handling Temporal Aspects

Temporal alignment was a **critical step** to ensure valid prediction.

- For each sample in the dataset, **features were taken from year *t***, and the model was trained to predict the **yield in year *t+1***.
- This alignment was achieved using '.shift(-1)' within each 'country-crop group'.
- By removing rows where the target value was missing (i.e., the last year per group), we preserved **temporal causality** — ensuring the model never saw future data during training.

This setup supports the real-world scenario where we make yield predictions for the upcoming year using only data available from the current year.



Graph 1: This scatter plot compares actual vs. predicted crop yields (2011–2022), with the red dashed line indicating perfect prediction

Section D: Preprocessing

In this section, we describe all preprocessing steps taken to prepare the dataset for training the multilayer perceptron model. These steps span across raw data loading, merging, spatial mapping, temporal aggregation, and final dataset construction. Each action was taken with a clear rationale to ensure consistency, usability, and predictive signal in the final data.

1. Data Cleaning and Standardization

All 17 raw CSV files were loaded using pandas, and column names were normalized to lowercase and stripped of whitespace to ensure consistency across files. For spatial matching and aggregation, latitude and longitude columns were rounded to one decimal place to match the spatial resolution of different sources. Rows with missing or non-finite coordinates were dropped to avoid errors during spatial operations.

2. Merging Multisource Environmental Data

From 14 environmental datasets (e.g., rainfall, soil moisture, soil temperature, canopy, evapotranspiration), we retained those that contained latitude, longitude, and year for compatibility. These were merged on the keys (latitude, longitude, year) using an outer join to retain maximum coverage. This resulted in a spatial-temporal dataset of gridded environmental variables.

3. Monthly Aggregation to Annual Averages

Many environmental variables were recorded monthly (e.g., rainf_tavg_month_1, ..., rainf_tavg_month_12). For each such group of 12 columns, we computed a single annual average feature using mean(axis=1), significantly reducing dimensionality and improving model generalizability. The original monthly columns were dropped post-aggregation. This reduced noise and potential overfitting while preserving temporal trends.

4. Country Assignment via Nearest Centroid Matching

Environmental and land cover data were initially in gridded format. To aggregate them by country and year, we used a KDTree-based nearest neighbour approach. Each grid cell was matched to the nearest country centroid from a country latitude/longitude lookup table. This allowed mapping of spatially resolved data to country-level granularity.

5. Land Cover Aggregation

Land cover data, originally in grid-cell format, was averaged by country and year. This produced 17 land cover class percentage features for each country-year pair (e.g., land_cover_percent_class_1, ..., land_cover_percent_class_17). This ensured compatibility with other country-level features and made it possible to link environmental and land cover conditions.

6. Environmental Aggregation by Country-Year

After country mapping, the environmental dataset was also grouped by country and year, and all features were averaged. This ensured that environmental variables and land cover variables were aligned in spatial and temporal resolution, and allowed merging with the yield data, which is available at the country-year level.

7. Yield Data Extraction and Formatting

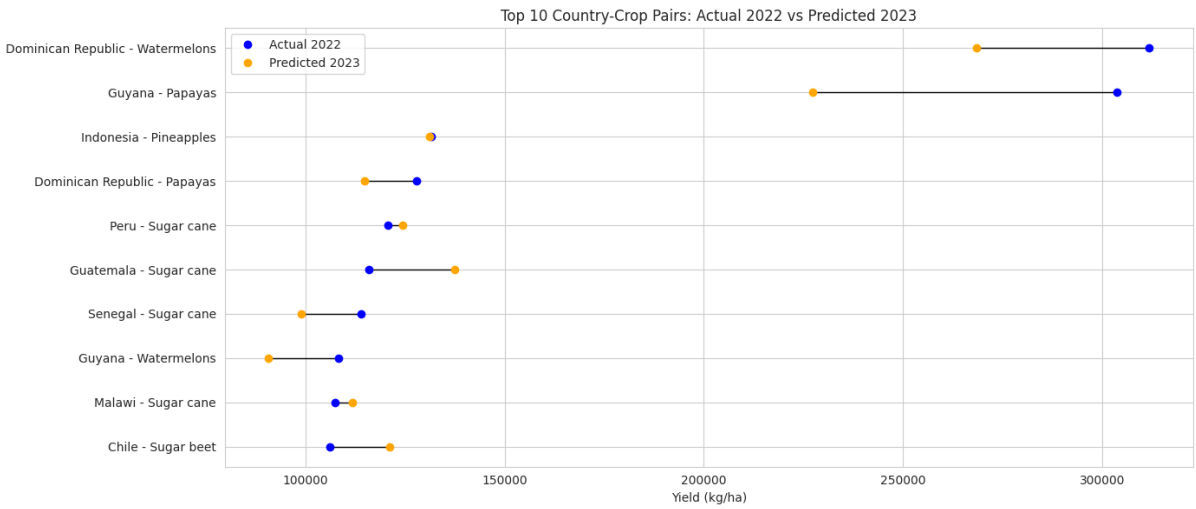
From the agricultural production dataset, only rows with the element "Yield" were retained. The resulting subset contained four columns: country, crop, year, and yield. This table was merged with the previously combined environmental and land cover dataset, using an inner join on country and year.

8. Final Dataset Construction

The merged dataset (final_df) contains **37 columns** and **67,963 rows**, with no missing values. It includes:

- **34 continuous input features** (17 environmental + 17 land cover).
- **2 identifiers**: country, crop.
- **1 temporal variable**: year.
- **1 output label**: yield.

All categorical fields (country, crop) were retained for potential one-hot encoding later in the modeling pipeline.



Graph 2: This dot plot compares actual 2022 yields with predicted 2023 yields across the top 10 country–crop pairs, highlighting deviations in model forecasts.